Abril 21, 2026

Estructuras de Datos

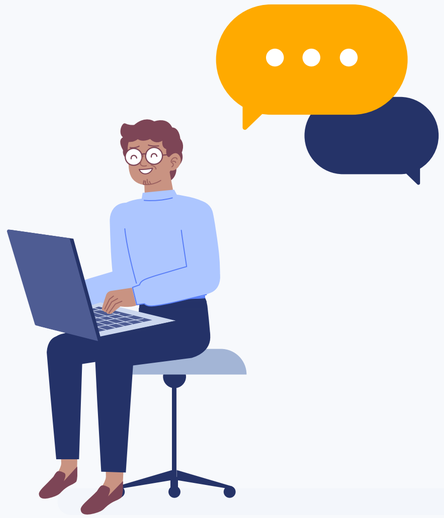
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Recordemos

Conceptos vistos en la anterior clase



Introducción a Listas Circulares

Ventajas y Desventajas de Listas Circulares

Ejercicios con Listas Circulares

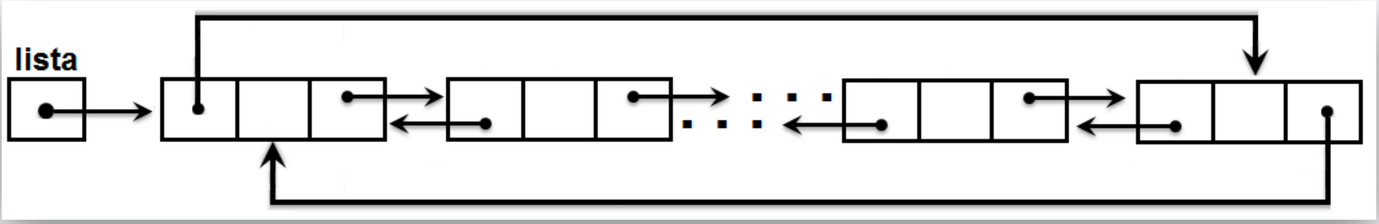
ESTRUCTURAS SECUENCIALES

Introducción a Listas Circulares Dobles

Que son Listas Circulares Dobles ?

- Una lista circular doble es una estructura de datos en la que cada nodo tiene **dos punteros**, uno que representa la posición del nodo anterior y otro del nodo siguiente. Esto significa que la lista es circular, ya que el último nodo apunta al primer nodo.

Representación de Listas Circulares Dobles



ESTRUCTURAS SECUENCIALES

Ventajas de Listas Circulares Dobles

Ventajas de Listas Circulares Dobles

- **Recorrido bidireccional:** significa que se puede recorrer en ambas direcciones, desde el principio hasta el final o desde el final hasta el principio, lo que es muy útil en muchas aplicaciones.

- **Implementación más sencilla de algunas operaciones:** Algunas operaciones, como la reversión de la lista, son más sencillas de implementar en una lista circular doble que una lista simplemente.

Ventajas de Listas Circulares Dobles

- **Mayor seguridad en el acceso a los nodos:** permiten que el acceso a los nodos sea más seguro, ya que se puede comprobar si un puntero es diferente de nulo antes de intentar acceder al nodo al que apunta.

Desventajas de Listas Circulares Dobles

Desventajas de Listas Circulares Dobles

- **Mayor complejidad en la implementación:** esto se debe a que cada nodo debe tener dos punteros, uno al nodo anterior y otro al nodo siguiente. Esto aumenta la complejidad de la estructura y puede aumentar el tiempo de desarrollo.

- **Mayor uso de memoria:** Debido a que cada nodo debe tener dos punteros, esta estructura puede utilizar más memoria que otras estructuras de datos.

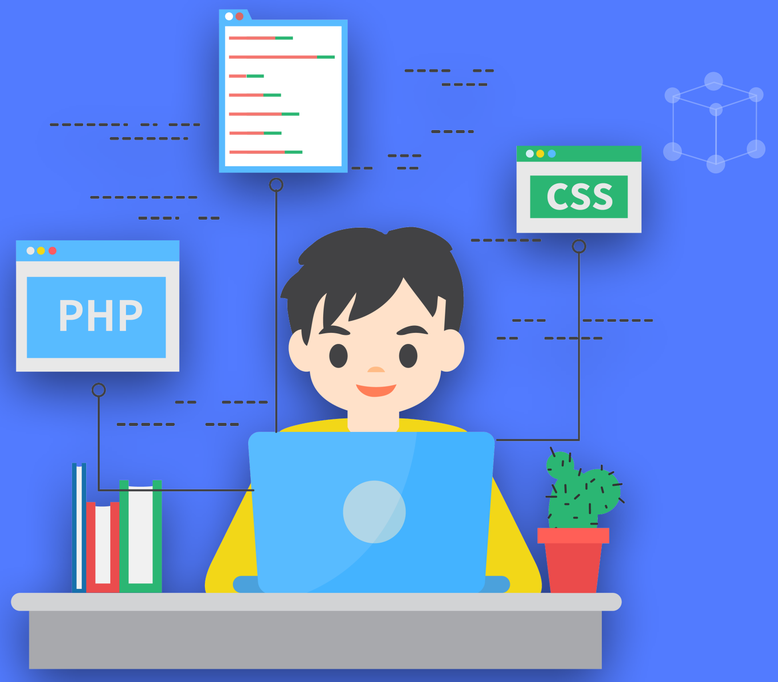
Desventajas de Listas Circulares Dobles

- **Mayor complejidad en el manejo de nodos:** La presencia de dos punteros en cada nodo aumenta la complejidad de lista. Por ejemplo, puede ser más difícil de manejar en ciertos algoritmos de clasificación o búsqueda de datos.

- **Mayor complejidad en la manipulación de nodos:** puede ser más difícil de eliminar nodos específicos en una lista ya que se debe re-asignar los punteros de siguiente y anterior.

LISTAS CIRCULARES DOBLES

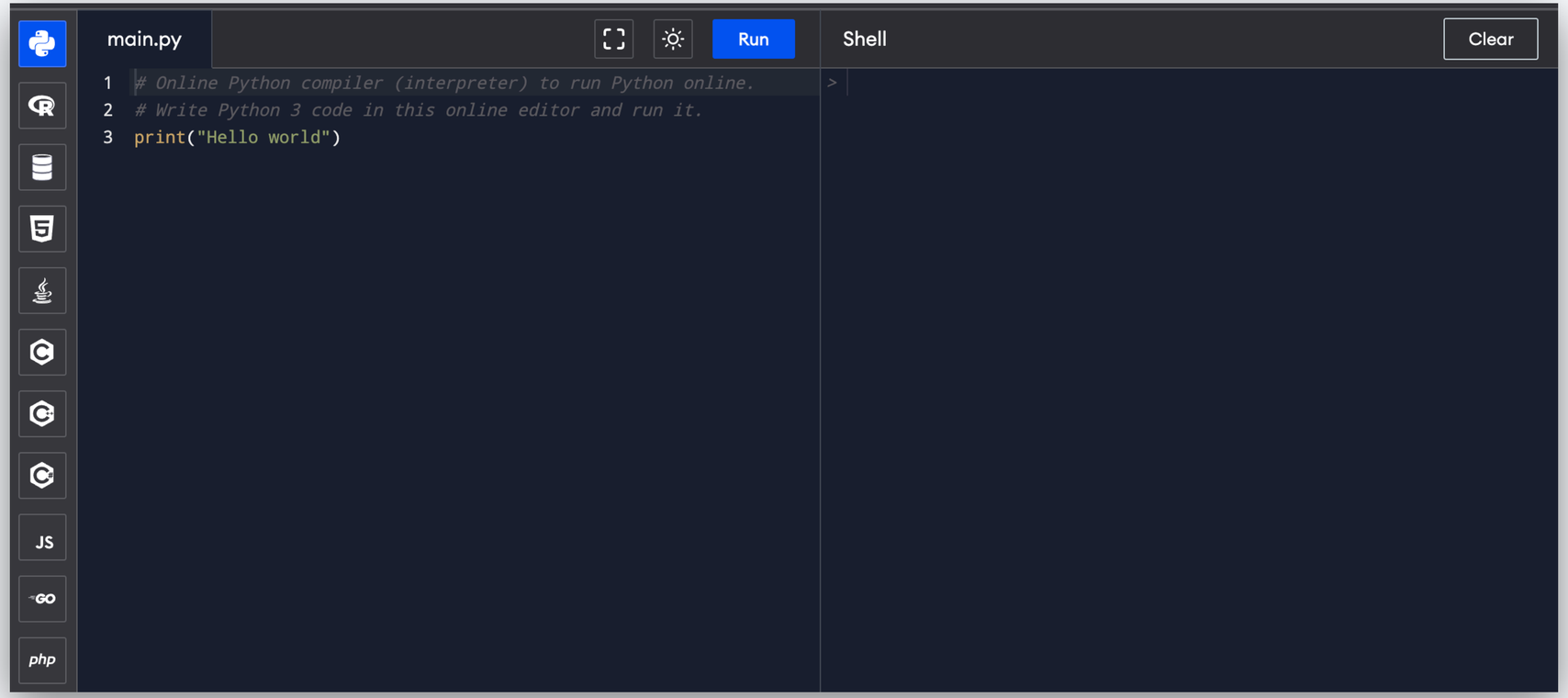
Practica con Python



Recursos IDE

Programiz

https://www.programiz.com/python-programming/online-compiler/



Código de la clase
(parte 1)

```
1   class Node:
2       def __init__(self, data):
3           self.data = data
4           self.prev = None
5           self.next = None
```

Código de la clase

(parte 1)

```
7   class CircularDoublyLinkedList:
8       def __init__(self):
9           self.head = None
10
11      # Insertar elementos al inicio de la lista
12  >   def insert_at_beginning(self, data):…
25
26      # Insertar elementos al final de la lista
27  >   def insert_at_end(self, data):…
39
40      # Eliminar elementos de la lista
41  >   def delete(self, data):…
70
71      # Impimir lista
72  >   def print_list(self):…
```

Código de la clase
(parte 1)

```
    # Insertar elementos al inicio de la lista
    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            new_node.prev = self.head
        else:
            last = self.head.prev
            new_node.next = self.head
            self.head.prev = new_node
            new_node.prev = last
            last.next = new_node
            self.head = new_node
```

Código de la clase
(parte 1)

```
    # Insertar elementos al final de la lista
    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            new_node.prev = self.head
        else:
            last = self.head.prev
            last.next = new_node
            new_node.next = self.head
            new_node.prev = last
            self.head.prev = new_node
```

LISTAS CIRCULARES DOBLES

Código de la clase

(parte 1)

```python
    # Eliminar elementos de la lista
    def delete(self, data):
        if self.head is None:
            print("La lista está vacía.")
            return

        current = self.head
        if current.data == data:
            if current.next == self.head:
                self.head = None
            else:
                last = self.head.prev
                self.head = current.next
                self.head.prev = last
                last.next = self.head
            return

        while current.next != self.head:
            if current.data == data:
                current.prev.next = current.next
                current.next.prev = current.prev
                return
            current = current.next

        if current.data == data:
            current.prev.next = self.head
            self.head.prev = current.prev
            return
```

Código de la clase
(parte 1)

```
    # Impimir lista
    def print_list(self):
        current = my_list.head
        if current is not None:
            print(current.data)
            current = current.next
        while current != my_list.head:
            print(current.data)
            current = current.next
```

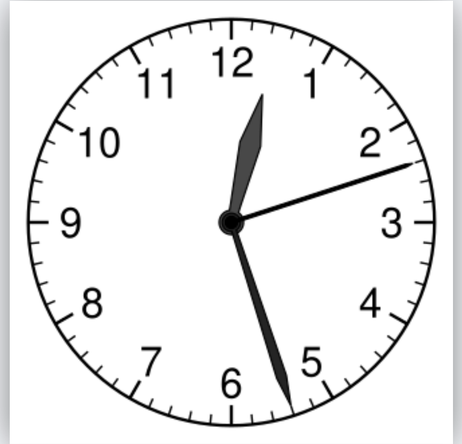
Código de la clase
(parte 1)

```
# Crear una instancia de la lista circular doblemente enlazada
my_list = CircularDoublyLinkedList()


# Insertar elementos al inicio y final de la lista
my_list.insert_at_beginning(3)
my_list.insert_at_end(5)
my_list.insert_at_beginning(1)
my_list.insert_at_end(7)


# Imprimir los elementos de la lista
my_list.print_list()


# Eliminar elementos de la lista
my_list.delete(3)
my_list.delete(7)


# Imprimir los elementos de la lista después de eliminar algunos
my_list.print_list()

```

LISTAS DOBLES

Taller Listas Circulares Dobles

TALLER

Desarrollo de un Reloj con Python

- Para este desarrollo solo debe utilizar Python
- La app debe tener Frontend
- Aumentar una funcionalidad que usted considere pertinente.
- Fecha de entrega: 28 de Abril.